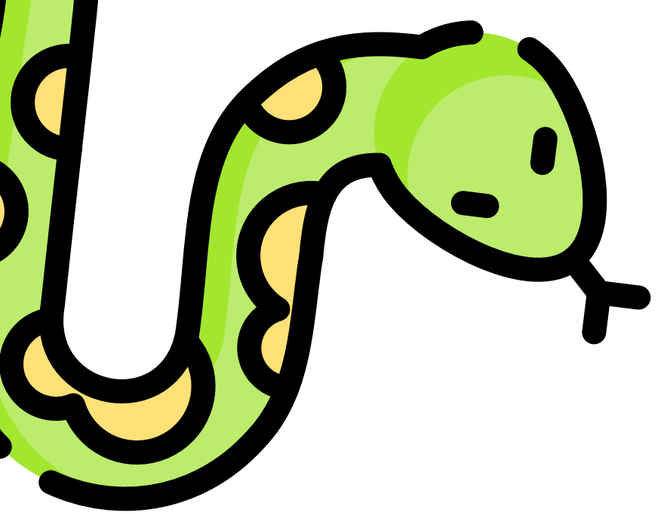
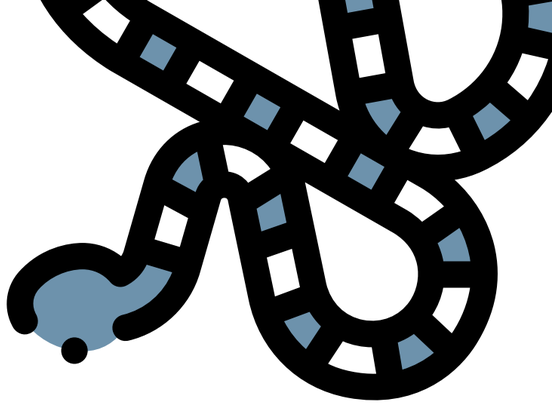


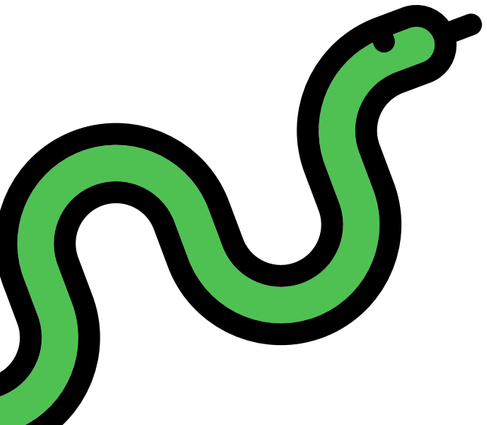


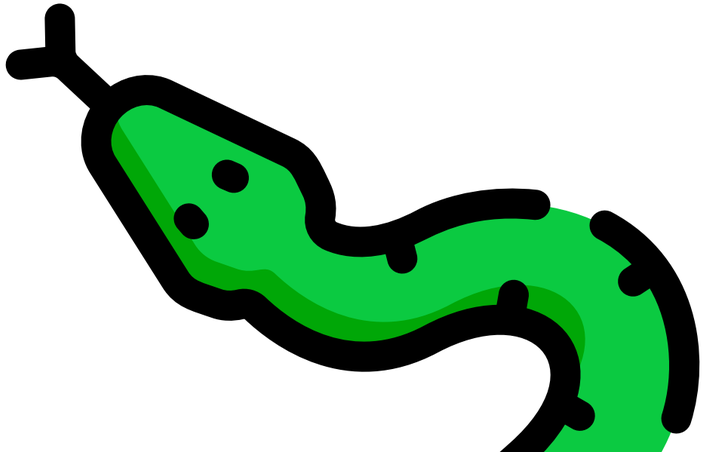
Snake-game

Implementation on Assembler & Logisim

the project of **Svirsky Bogdan**, **Ilya Pereverzev** and **Grigory Chernikov**

Gr. 23215

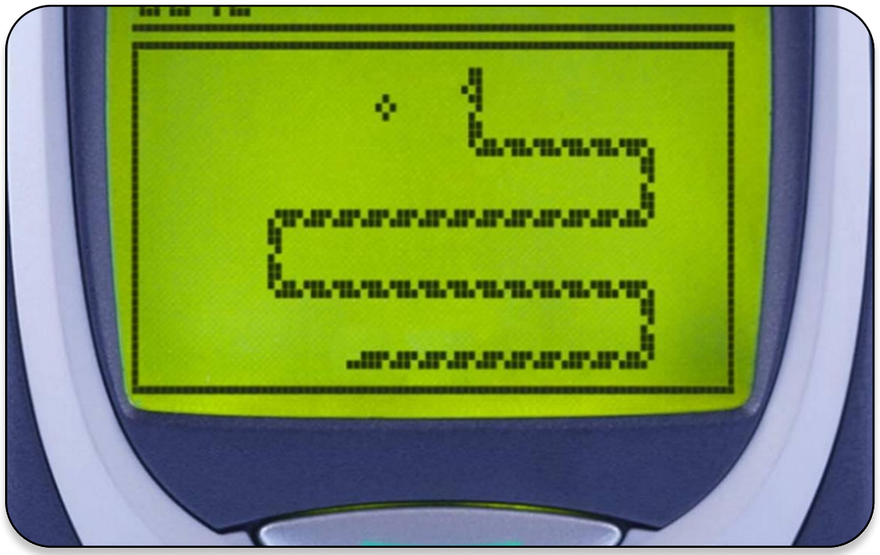




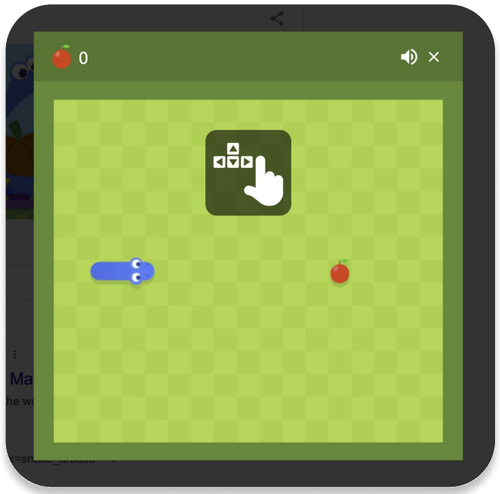
What is a "Snake game"?

General rules:

- the snake moves on display in **4 directions**
- the snake can **eat food** and **increase in size**
- the user loses when the snake **crashes into itself** or into walls
- the user wins when earns the **right amount of points**



Snake game on Nokia



Snake game

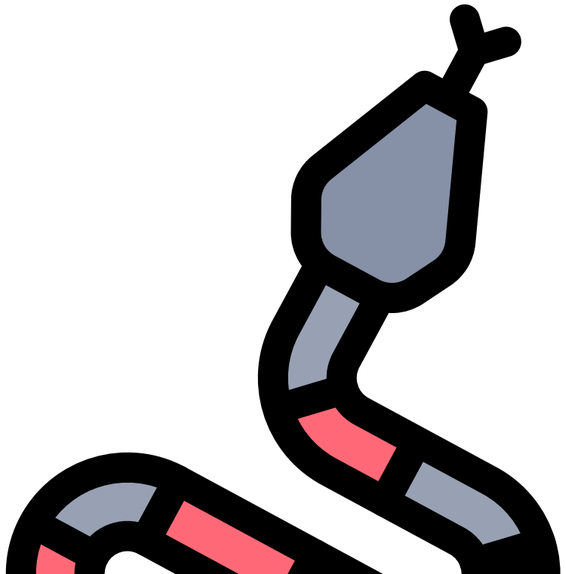
Slither.io

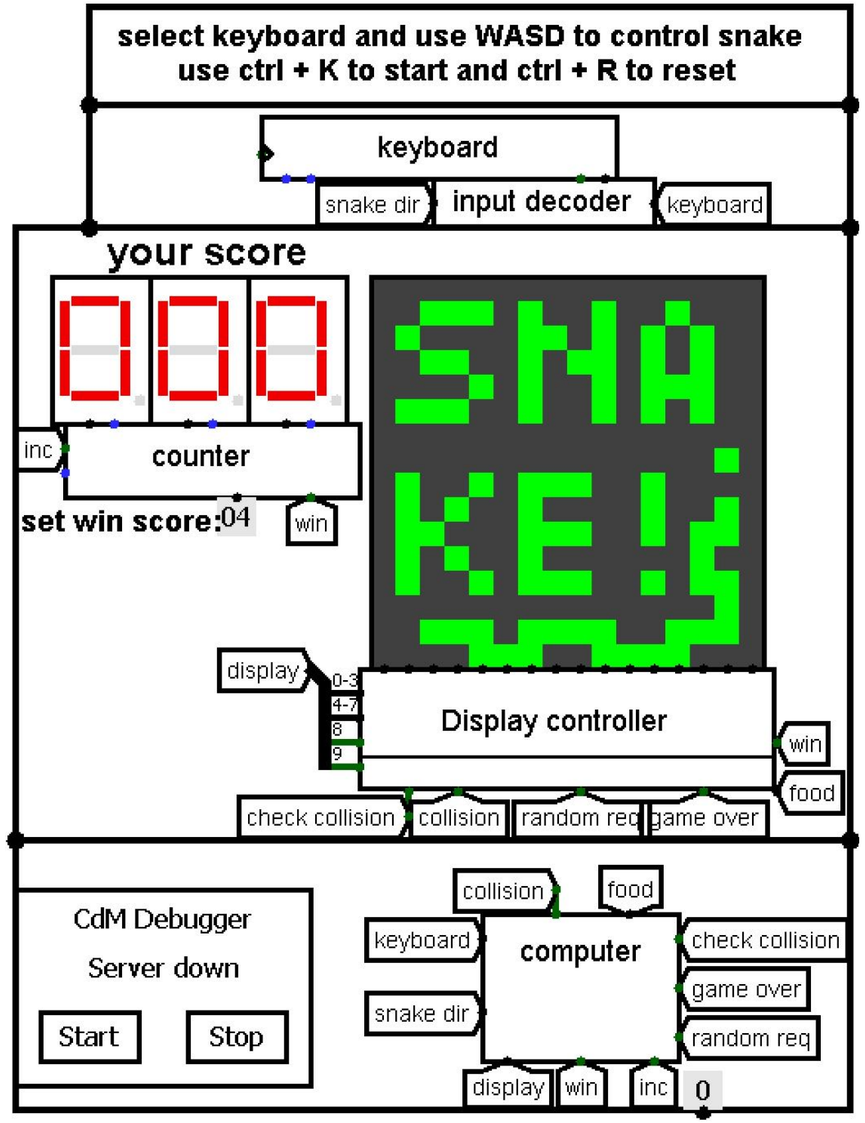
How did we do it?

Hand-made circuits:

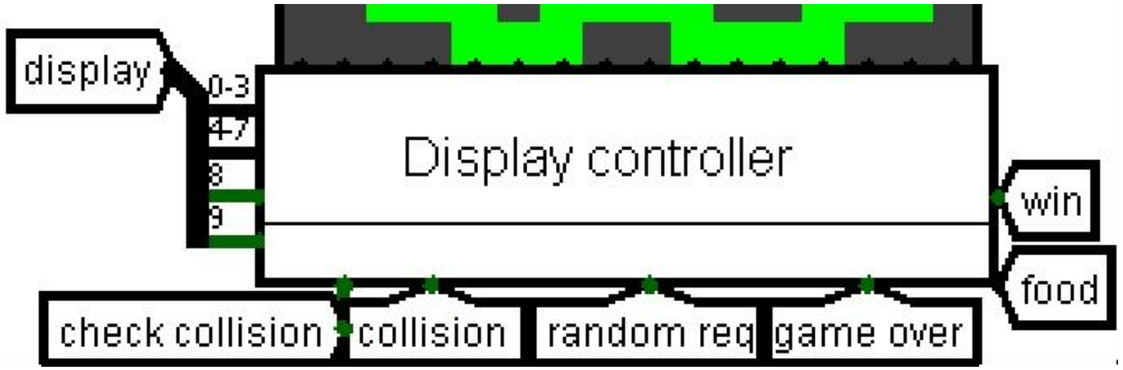
- display controller
- counter
- input decoder
- memory map extrude (hidden in computer)
- etc. (small but useful circuits)

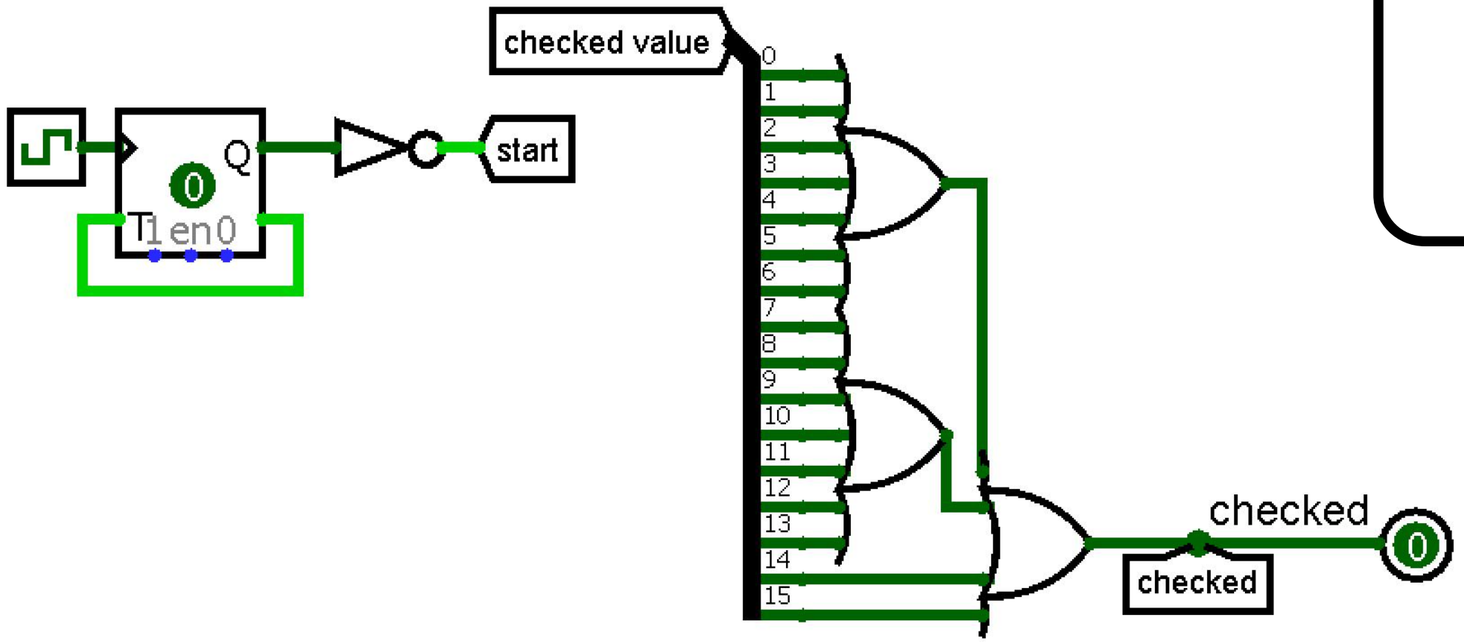
And 295 lines of Assembler code





How does display controller work?

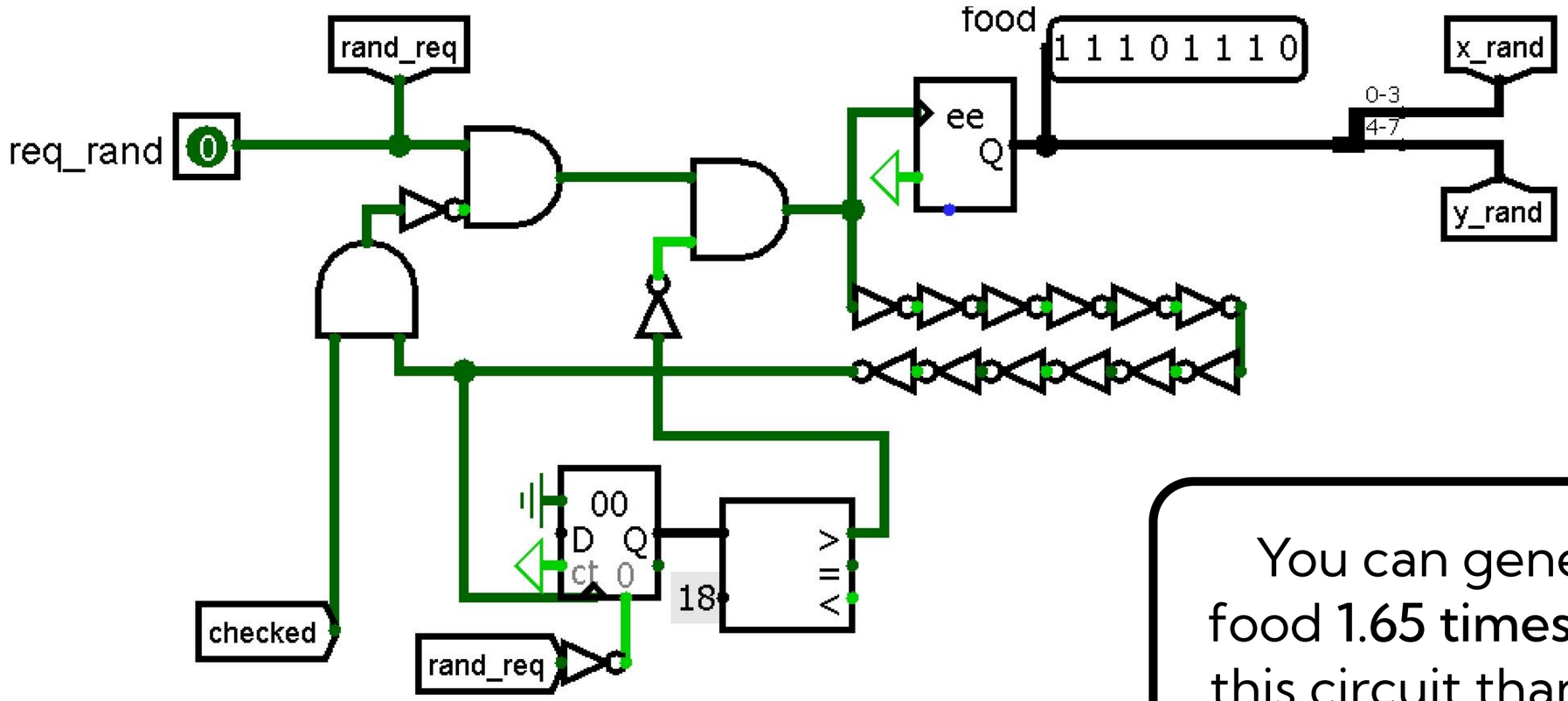




Data to display:

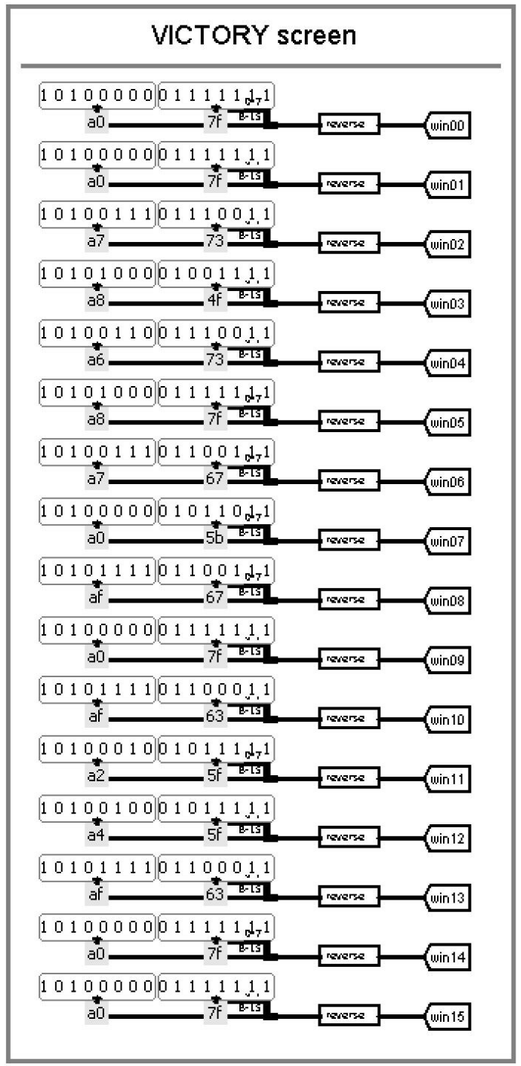
- bit 0–3 – X coordinate
- bit 4–7 – Y coordinate
- bit 8 – pixel state
- bit 9 – write

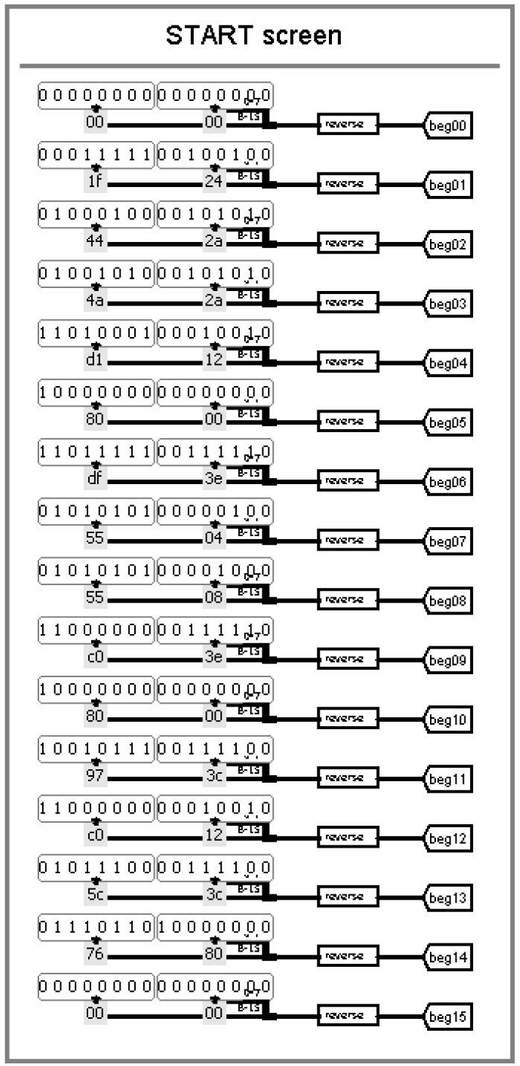
How does display controller work?

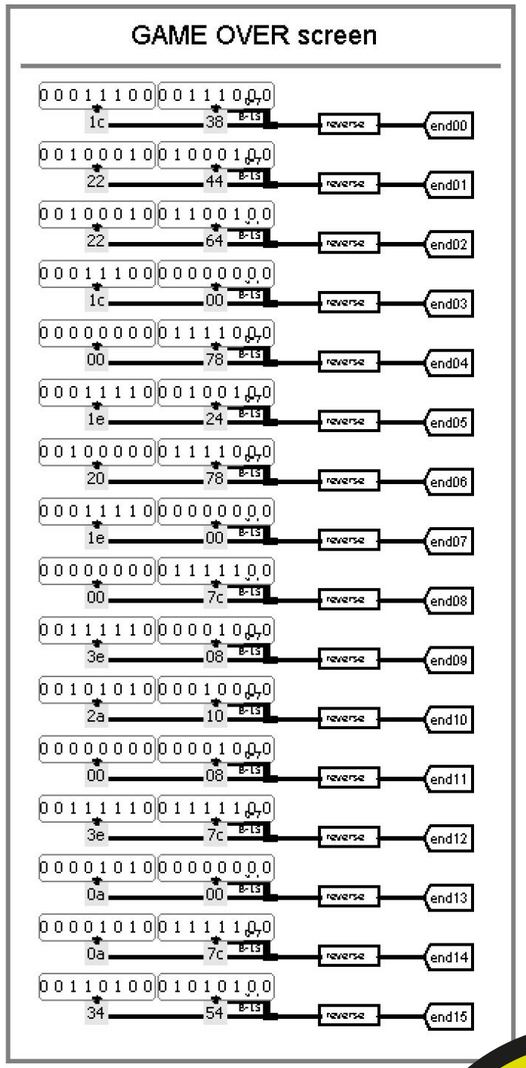


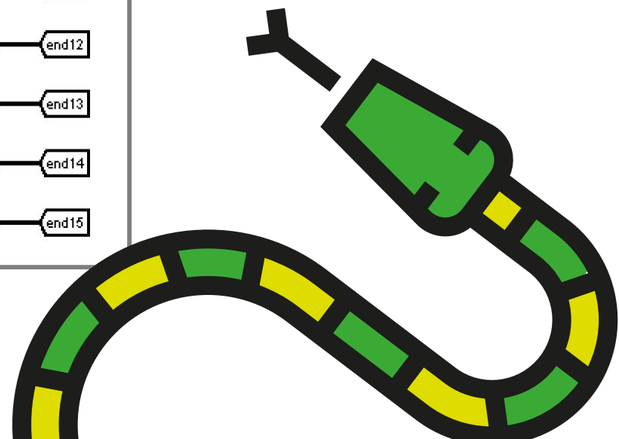
You can generate new food **1.65 times faster** with this circuit than by using a simple random generator

How does display controller works?

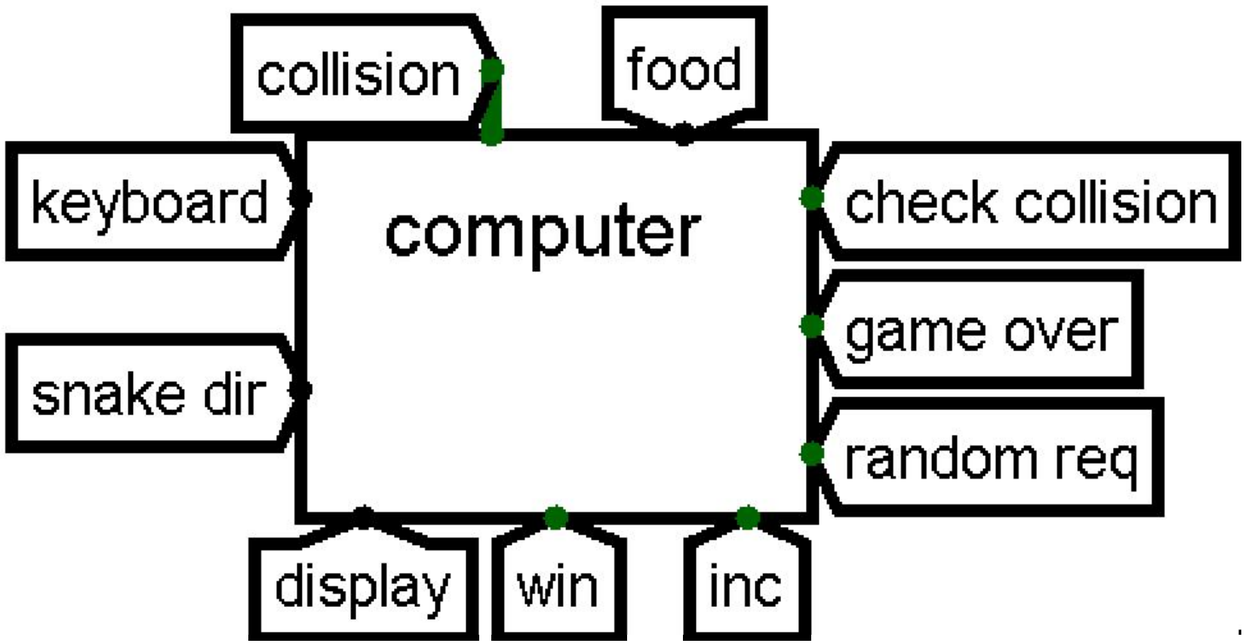




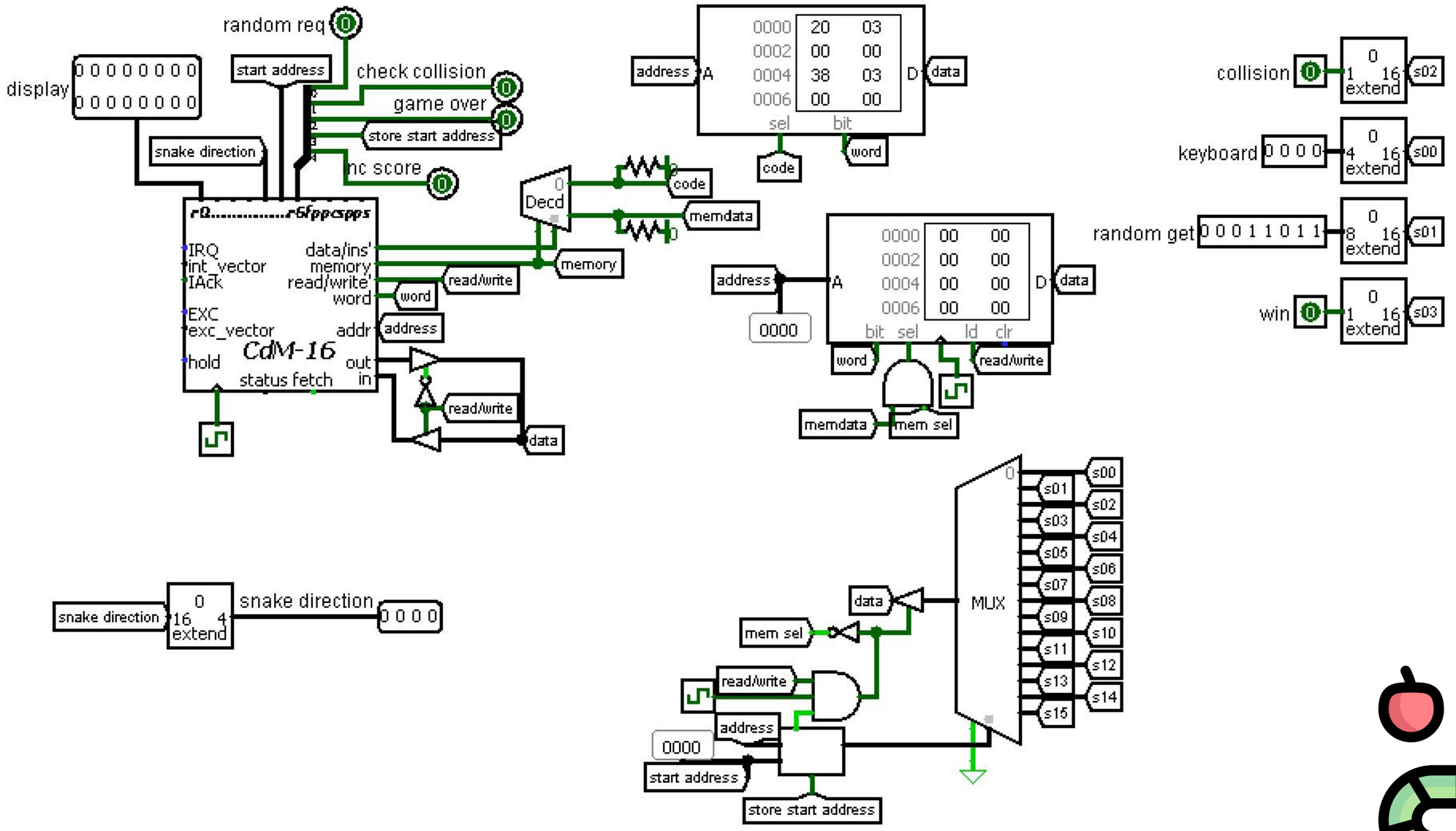




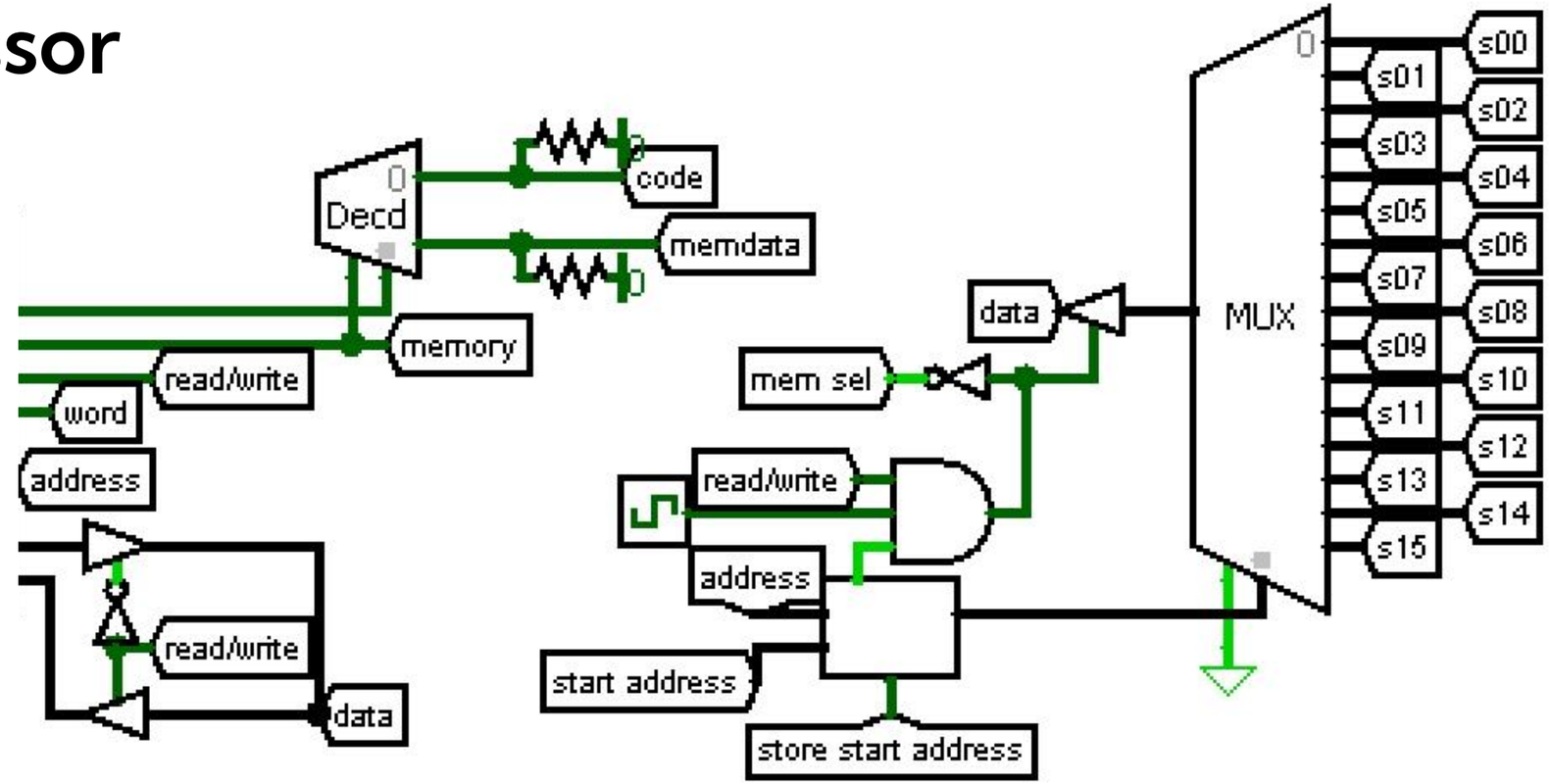
What about the computer?

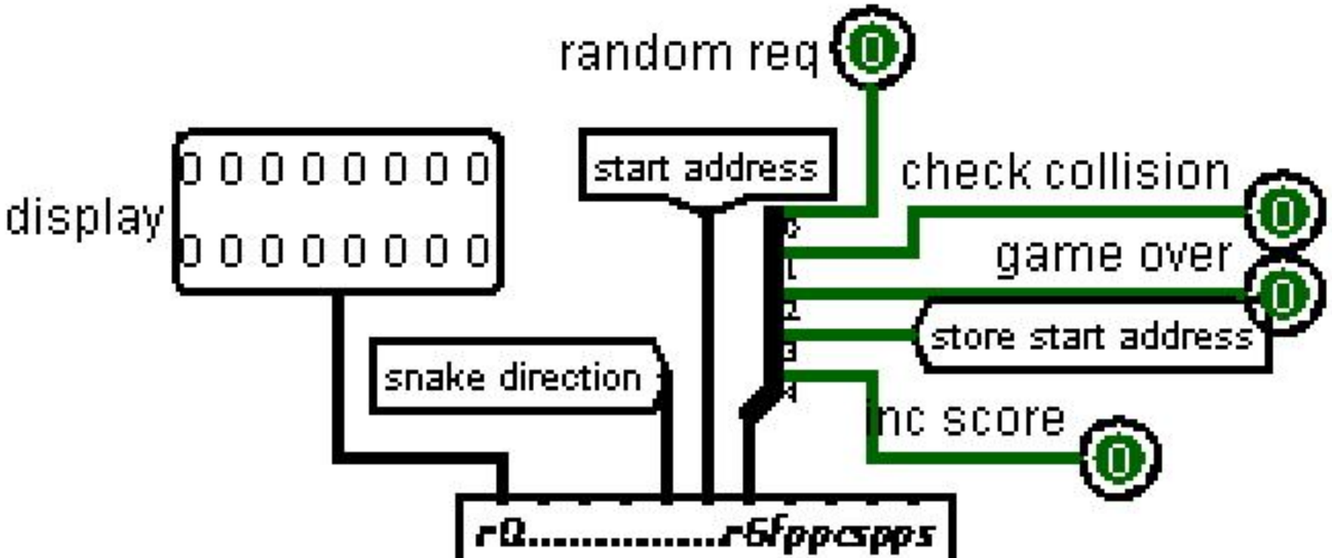


What's inside?



How does the processor interact with circuit?





We implemented memory mapped input in our circuit

What does it look like in code?

```
external_devices_data: # allocating bytes for memory mapped I/O
    ds 16
```

And we can use simple instruction to load data from external devices

```
ldi r1, external_devices_data + 2
ldb r1, r1 # getting result from display controller

ldi r1, external_devices_data + 1
ldb r1, r0 # getting result of checking pixel
```

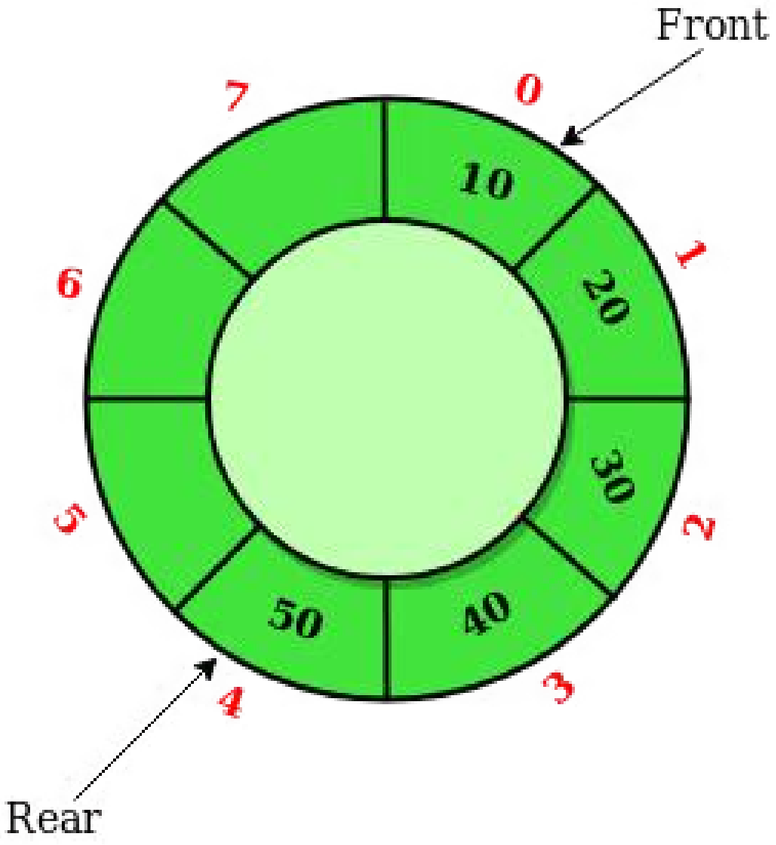
How did we implement the snake in code?

Queue data structure

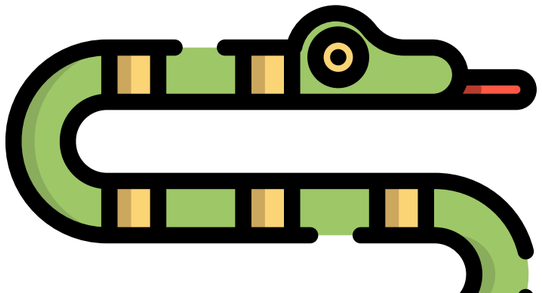
The lifecycle of the pixel in a snake is equal to the lifecycyle of elements in queue

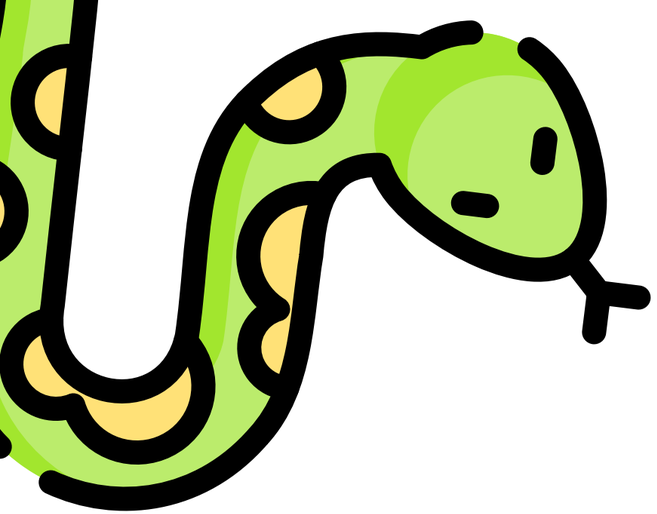
How did we implement the snake in code?

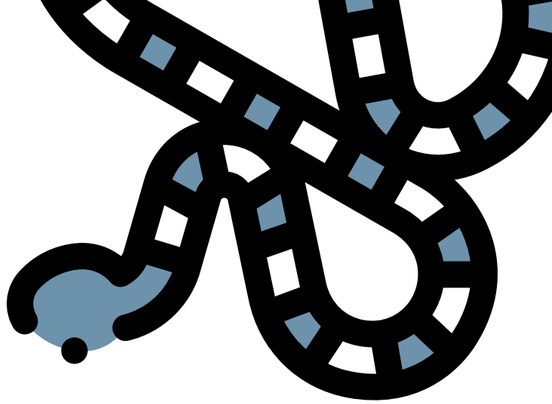
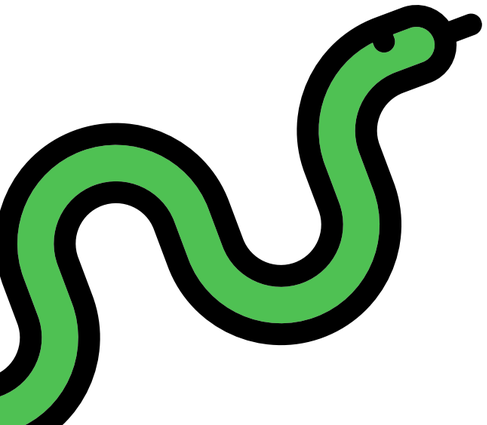
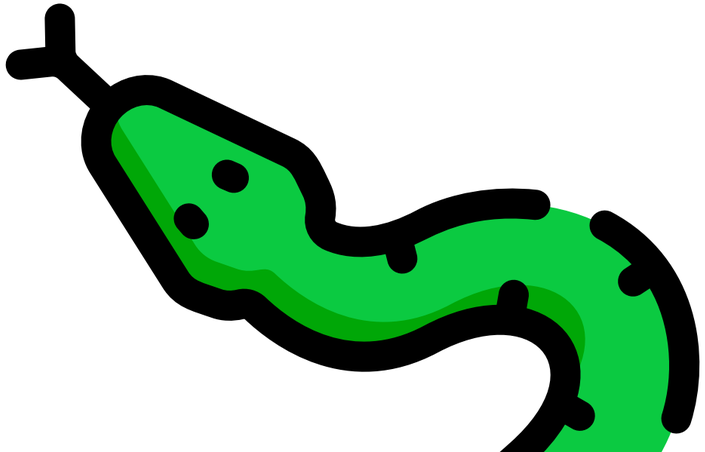
Circular buffer for queue implementation



Circular buffer is the most suitable for us, because we can allocate required amount of bytes in the beginning and quickly working with queue







Thanks for your attention!